BP 算法解决二分类问题

一、实验内容

本次实验我们尝试使用利用前馈神经网络解决 XOR 问题。我们搭建一个带单隐层的神经网络，用 BP（反向传播）算法完成参数更新，让网络能够把四组输入

$$X = \{[0,0],[0,1],[1,0],[1,1]\}$$

对应到标签

$$Y = \{0,1,1,0\}$$

上。这个任务数据量不大，但是正好能说明一个问题：像 XOR 这种线性不可分的数据，单层线性模型是处理不了的，必须通过隐藏层引入非线性变换。

本次实验我们的任务流程如下所示：我们规定迭代次数为 600 次，并使用 Adam 作为我们的优化器，选取学习率为 0.01 ，另外，我们选取 MSE 作为损失函数，对最终结果进行优化更新，另外，我们的神经网络架构如下所示：我们引入 torch 的 nn.Module 板块，设置输入层与输出层如下下所示：

```
self.layer1 = nn.Linear(3, 5)
self.lyaer2 = nn.Linear(5, 1)
```

并选取 Sigmoid 函数作为激活函数：

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

我们设置 batch_size=4 ，并得到了最后的网络架构。

二、神经网络简介

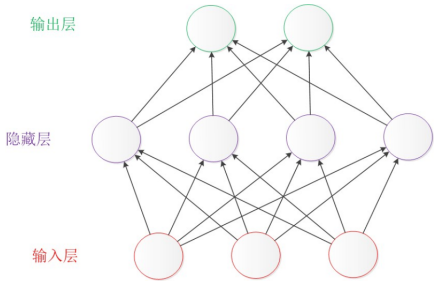
介绍

神经网络中的结构如下所示，并且他有着这样的解释：

1. 设计一个神经网络时，输入层与输出层的节点数往往是固定的，中间层则可以自由指定；
2. 神经网络结构图中的拓扑与箭头代表着预测过程时数据的流向，跟训练时的数据

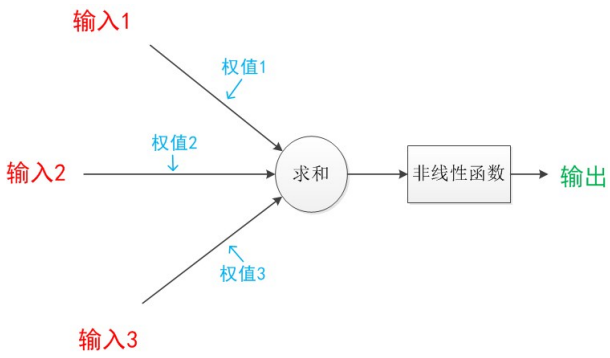
流有一定的区别

3. 结构图里的关键不是圆圈（代表“神经元”），而是连接线（代表“神经元”之间的连接）。每个连接线对应一个不同的权重（其值称为权值），这是需要训练得到的。



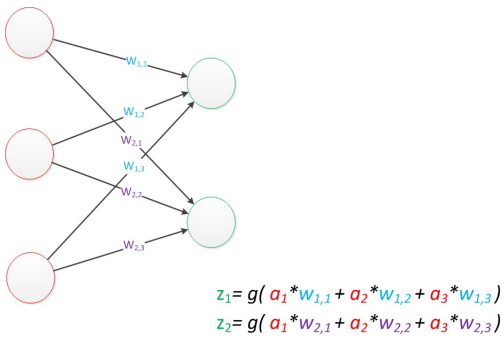
神经元

神经元模型是一个包含输入、输出和计算功能的模型，这里输入类比为树突，输出看成是轴突，计算可以类比为细胞核，其中计算可以被分为加权和非线性映射。



单层神经网络 (感知器)

当一个神经元的输出可以向多个神经元传递，则我们就产生了单层神经网络，我们可以用下面这张图来表示单层神经网络：如果我们用 $\boldsymbol{x} = [a_1, a_2, a_3]^T$ 表示输入，$\boldsymbol{z} =$



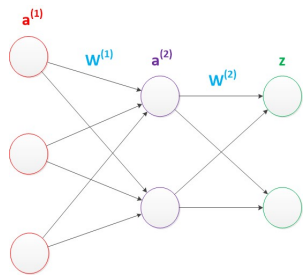
$[z_1, z_2]^T$ 表示输出，并且将系数写作矩阵的形式 W , 则有输出公式

$$\boldsymbol{z} = g(W\boldsymbol{a})$$

两层神经网络 (多层感知器)

基本概念

两层神经网络除了包含一个输入层，一个输出层以外，还增加了一个中间层。此时，中间层和输出层都是计算层。我们扩展上节的单层神经网络，在右边新加一个层次。



我们用举着你形式来表示上面的神经网络：

$$g(W^{(1)}\boldsymbol{a}^{(1)}) = \boldsymbol{a}^{(2)}$$

$$g(W^{(2)}\boldsymbol{a}^{(2)}) = \boldsymbol{z}$$

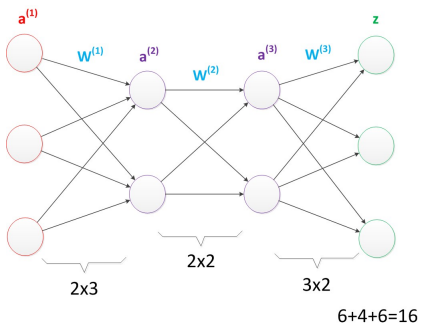
在单层神经网络中，我们一般使用 sgn 函数来作为非线性函数，而对于两层神经网络，我们选择 sigmoid 作为函数 g 。这里我们基于损失函数与梯度下降对权重进行训练，我们使用一定量的数据集，并对神经网络中的所有参数赋上随机值，我们采用求导的形式对我们使用的损失函数，一般是平方损失函数：

$$\text{loss} = ||y - y_p||^2$$

对每个参数采用梯度下降法，找到最优解，这便对神经网络完成了训练。

多层神经网络

对于两层神经网络，我们可以再增加几个中间层，进而就得到了多层神经网络。



更深入的表示特征可以这样理解，随着网络的层数增加，每一层对于前一层次的抽象表示更深入。在神经网络中，每一层神经元学习到的是前一层神经元值的更抽象的表示。例如第一个隐藏层学习到的是“边缘”的特征，第二个隐藏层学习到的是由“边缘”组成的“形状”的特征，第三个隐藏层学习到的是由“形状”组成的“图案”的特征，最后的隐藏层学习到的是由“图案”组成的“目标”的特征。通过抽取更抽象的特征来对事物进行区分，从而获得更好的区分与分类能力。

在多层神经网络中，我们一般采用 ReLU 函数作为非线性函数，其更容易收敛，并且预测性能较好。对于训练，我们采用的方式仍然是优化和泛化。

三、反向传播算法原理

本实验采用一个包含输入层、隐藏层和输出层的三层前馈神经网络来解决 XOR 二分类问题。网络输入为二维向量

$$\mathbf{x} = [x_1, x_2]^T,$$

输出为预测值 $\hat{y}$，真实标签记为 y。

设隐藏层包含 m 个神经元，则输入层到隐藏层的权重矩阵和偏置向量分别记为 $W^{(1)} \in \mathbb{R}^{m\times 2}$, $b^{(1)} \in \mathbb{R}^m$, 隐藏层到输出层的权重矩阵和偏置分别记为 $W^{(2)} \in \mathbb{R}^{1\times m}$, $b^{(2)} \in \mathbb{R}$

0.1 前向传播过程

对于任意输入样本 $\mathbf{x}$，首先计算隐藏层的线性组合：$z^{(1)} = W^{(1)}\mathbf{x} + b^{(1)}$, 然后经过 Sigmoid 激活函数，得到隐藏层输出：$a^{(1)} = \sigma(z^{(1)})$, 其中 Sigmoid 函数定义为

$$\sigma(t) = \frac{1}{1+e^{-t}}.$$

接着，输出层的线性组合为 $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$, 再经过 Sigmoid 激活函数，得到

网络最终输出：$\hat{y} = a^{(2)} = \sigma(z^{(2)})$. 因此，前向传播的计算过程可以概括为：

$$\mathbf{x} \to z^{(1)} \to a^{(1)} \to z^{(2)} \to \hat{y}.$$

0.2 损失函数

本实验采用均方误差作为损失函数。对于单个样本，其损失定义为

$$L = \frac{1}{2}(y - \hat{y})^2.$$

其中，y 为真实标签，$\hat{y}$ 为网络预测输出。引入系数 $\frac{1}{2}$ 的目的是在求导时抵消平方项带来的系数，简化计算过程。

0.3 反向传播过程

反向传播算法的核心思想是：先计算输出层误差，再利用链式法则将误差逐层向前传播，从而得到各层参数的梯度，并完成参数更新。

0.3.1 输出层误差项

输出层误差项定义为损失函数对输出层线性输入 $z^{(2)}$ 的偏导数，即

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}}.$$

由链式法则可得

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}}.$$

因为

$$L = \frac{1}{2}(y - \hat{y})^2,$$

所以

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y.$$

又由于输出层采用 Sigmoid 激活函数，因此

$$\frac{\partial \hat{y}}{\partial z^{(2)}} = \hat{y}(1 - \hat{y}).$$

故输出层误差项为

$$\delta^{(2)} = (\hat{y} - y)\hat{y}(1 - \hat{y}).$$

0.3.2 隐藏层误差项

隐藏层误差项定义为损失函数对隐藏层线性输入 $z^{(1)}$ 的偏导数，即

$$\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}}.$$

根据链式法则，隐藏层误差由输出层误差反向传递得到：

$$\delta^{(1)} = \left((W^{(2)})^T \delta^{(2)}\right) \odot a^{(1)}(1 - a^{(1)}),$$

其中 $\odot$ 表示对应元素相乘。

该公式表明，隐藏层每个神经元的误差由两部分共同决定：一部分来自输出层反向传回的误差信息，另一部分来自当前层激活函数的导数。

0.4 梯度计算

在得到各层误差项后，即可进一步求出各参数的梯度。

对于输出层参数，有

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)}(a^{(1)})^T, \ \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}.$$

对于输入层到隐藏层的参数，有

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)}\mathbf{x}^T, \ \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

上述公式给出了损失函数相对于各层权重和偏置的变化率，是后续参数更新的基础。

0.5 参数更新

设学习率为 η，则根据梯度下降法，各参数更新公式为

$$W^{(2)} \leftarrow W^{(2)} - \eta\frac{\partial L}{\partial W^{(2)}}, \ b^{(2)} \leftarrow b^{(2)} - \eta\frac{\partial L}{\partial b^{(2)}},$$

$$W^{(1)} \leftarrow W^{(1)} - \eta \frac{\partial L}{\partial W^{(1)}},\ b^{(1)} \leftarrow b^{(1)} - \eta \frac{\partial L}{\partial b^{(1)}}.$$

通过不断重复“前向传播—损失计算—反向传播—参数更新”的过程，网络参数会逐渐逼近最优值，从而使模型能够正确学习 XOR 问题中的非线性映射关系。

0.6 算法流程总结

对于训练集中的每一个样本，反向传播算法的执行步骤如下：

1. 将输入样本送入网络，计算隐藏层输出和最终预测值；
2. 根据预测值与真实值计算损失函数；
3. 计算输出层误差项 $\delta^{(2)}$；
4. 由输出层误差反向计算隐藏层误差项 $\delta^{(1)}$；
5. 根据误差项求出各层权重和偏置的梯度；
6. 按照梯度下降公式更新网络参数；
7. 重复上述过程，直到损失收敛或达到预设迭代次数。

四、问题描述与原理

XOR 的特点是：两个输入不同时输出 1，相同时输出 0。它看起来很简单，但不能直接用一条直线分开，所以很适合作为 BP 神经网络的入门例子。

我这里使用的是一个三层结构的网络：输入层、单隐层和输出层。因为代码里额外拼接了一列常数 1 来模拟偏置，所以输入维度实际按 3 维处理。隐藏层有 5 个神经元，激活函数采用 sigmoid，输出层输出 1 个值。损失函数用的是均方误差（MSE），优化器使用 Adam。

整个前向传播过程可以写成：

$$H = \sigma(XW_1 + b_1), \qquad \hat{Y} = HW_2 + b_2$$

其中 $\sigma(\cdot)$ 表示 sigmoid 函数。训练时通过反向传播计算梯度，再不断更新参数，使预测值 $\hat{Y}$ 逐步接近真实标签 Y。

五、实验步骤

1. 构造 XOR 数据集，并把输入和标签都转成张量；
2. 为了方便处理偏置，在输入后额外拼接一列常数 1；
3. 定义网络结构：输入层 3 维，隐藏层 5 维，输出层 1 维；
4. 编写前向传播函数，隐藏层使用 sigmoid 激活；
5. 设定损失函数为 MSELoss，优化器为 Adam，学习率设为 0.01；
6. 训练 600 轮，并记录每轮的损失值；
7. 在训练结束后，对四组 XOR 输入进行预测，并将结果可视化展示。

六、核心代码

下面是本次实验使用的主要代码。为了和运行结果保持一致，这里保留了我实际写程序时的网络结构和训练参数。Code of this part is available at [1]

```python
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt

EPOCH = 600
LR = 0.01

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

x_all = torch.tensor([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=torch.
    float32).to(device)
y_all = torch.tensor([[0], [1], [1], [0]], dtype=torch.float32).to(device
    )

dataset = TensorDataset(x_all, y_all)
dataloader = DataLoader(dataset, batch_size=4, shuffle=True)

```

[1] https://github.com/HaoranYuandomestic/ex1_BP_algorithm.git

```python
class LinearModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(3, 5)
        self.layer2 = nn.Linear(5, 1)

    def forward(self, x):
        tmp = torch.sigmoid(self.layer1(x))
        return self.layer2(tmp)

model = LinearModel().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
loss_history = []

for epoch in range(EPOCH):
    for x_batch, y_batch in dataloader:
        x_batch = torch.cat((x_batch, torch.ones(x_batch.size(0), 1).to(device)), dim=1)
        y_pred = model(x_batch)
        loss = criterion(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        loss_history.append(loss.item())

    if (epoch + 1) % 100 == 0:
    print(f"Epoch [{epoch + 1}/{EPOCH}], Loss: {loss.item():.4f}")

print("The training has finished.")
with torch.no_grad():
x_all = torch.cat((x_all, torch.ones(x_all.size(0), 1).to(device)), dim=1)
y_pred = model(x_all)
print("Predicted outputs:")
print(y_pred.cpu().numpy())
```

七、实验结果与分析

从训练结果来看，模型最后是能够把 XOR 这四组样本分清楚的。训练初期损失下降比较快，后面在 200 轮左右有一段比较平缓的阶段，再继续训练后，损失逐渐逼近 0，说明网络已经基本收敛。

如果把预测结果按 0/1 来看，四组输入对应的输出和真实值是一致的，也就是：

$$[0,0] \to 0, \quad [0,1] \to 1, \quad [1,0] \to 1, \quad [1,1] \to 0$$

这说明加入隐藏层以后，网络已经具备处理线性不可分问题的能力。

在本次实验中，我们尝试解释了神经网络除去线性层外激活函数的作用，如果没有激活函数神经网络本质上仍然是一个线性加权函数，不管怎么调参数都很难得到正确分类；但只要加上隐藏层和激活函数，问题一下子就能做出来。

另外，从损失曲线也能看出来，训练并不是一直匀速下降的，中间会有一段波动或者平台期。这其实也挺正常，和初始化参数、学习率、优化器选择都有关系。这里用 Adam 之后，整体训练还是比较稳定的。

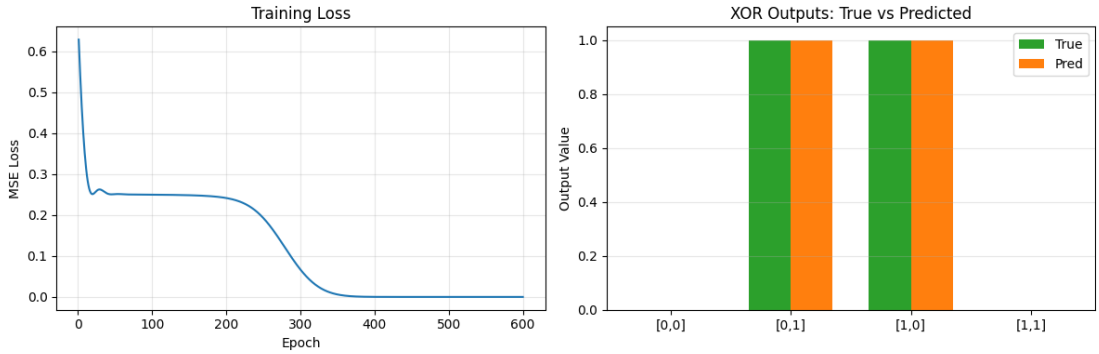


图 1: 训练损失曲线与 XOR 预测结果对比

八、实验总结

在本次实验中，我们实现了前馈神经网络最基本的架构：数据输入、网络搭建、前向传播、损失计算、反向传播、参数更新以及结果可视化。并直观的感受神经网络在分类问题的强大之处。另外，我们将数据具体化，以数据体现隐藏层的作用。